

```
1 import os
2 import time
3 import warnings
4 from dataclasses import dataclass
5 from datetime import datetime
6 from typing import Optional
7
8 import numpy as np
9 import pandas as pd
10 import yfinance as yf
11
12 warnings.filterwarnings("ignore")
13
14
15 @dataclass
16 class TradeSignal:
17     ticker: str
18     signal: str
19     price: float
20     confidence: int
21     stop_loss: Optional[float]
22     take_profit: Optional[float]
23     timestamp: str
24
25
26 class TradingBot:
27     """Technical analysis bot using SMA, RSI, MACD, Bollinger Bands, and ATR."""
28
29     def __init__(self, name: str = "Technical Analysis Bot"):
30         self.name = name
31         print(f"{self.name} initialized.\n")
32
33     # ----- #
34     # Data #
35     # ----- #
36
37     def fetch_data(
38         self,
39         ticker: str = "BTC-USD",
40         period: str = "60d",
41         interval: str = "1h",
42     ) -> pd.DataFrame:
43         data = yf.download(ticker, period=period, interval=interval, progress=False)
44         if data.empty:
45             raise ValueError(
46                 f"No data fetched for {ticker}. Check the ticker symbol or your internet connection."
47             )
48         # Recent yfinance versions return MultiIndex columns for single-ticker downloads
49         if isinstance(data.columns, pd.MultiIndex):
50             data.columns = data.columns.get_level_values(0)
51         data["Returns"] = data["Close"].pct_change()
52         return data
53
```

```
54 # ----- #
55 # Indicators #
56 # ----- #
57
58 def calculate_indicators(self, df: pd.DataFrame) -> pd.DataFrame:
59     df = df.copy()
60
61     # Moving averages
62     df["SMA_20"] = df["Close"].rolling(window=20).mean()
63     df["SMA_50"] = df["Close"].rolling(window=50).mean()
64
65     # RSI - clip() is cleaner than where() and .replace(0) prevents ÷0
66     delta = df["Close"].diff()
67     gain = delta.clip(lower=0).rolling(window=14).mean()
68     loss = (-delta.clip(upper=0)).rolling(window=14).mean()
69     rs = gain / loss.replace(0, np.nan)
70     df["RSI"] = 100 - (100 / (1 + rs))
71
72     # MACD - column named Signal_Line to avoid collision with trade signal strings
73     ema_12 = df["Close"].ewm(span=12, adjust=False).mean()
74     ema_26 = df["Close"].ewm(span=26, adjust=False).mean()
75     df["MACD"] = ema_12 - ema_26
76     df["Signal_Line"] = df["MACD"].ewm(span=9, adjust=False).mean()
77
78     # Bollinger Bands
79     bb_mid = df["Close"].rolling(window=20).mean()
80     bb_std = df["Close"].rolling(window=20).std()
81     df["BB_Mid"] = bb_mid
82     df["BB_Upper"] = bb_mid + 2 * bb_std
83     df["BB_Lower"] = bb_mid - 2 * bb_std
84
85     # ATR
86     tr = pd.concat(
87         [
88             df["High"] - df["Low"],
89             (df["High"] - df["Close"].shift()).abs(),
90             (df["Low"] - df["Close"].shift()).abs(),
91         ],
92         axis=1,
93     ).max(axis=1)
94     df["ATR"] = tr.rolling(window=14).mean()
95
96     # 5-period price momentum
97     df["Momentum"] = df["Close"].pct_change(5)
98
99     return df.dropna()
100
101 # ----- #
102 # Signals #
103 # ----- #
104
105 def score_signal(self, row: pd.Series) -> int:
106     """Score bullish technical conditions on a 0-100 scale. Fully deterministic."""
```

```
107     score = 0
108     if row["Close"] > row["SMA_20"] > row["SMA_50"]:
109         score += 35
110     if row["RSI"] < 30:
111         score += 25
112     elif row["RSI"] < 50:
113         score += 10
114     if row["MACD"] > row["Signal_Line"]:
115         score += 25
116     if row["Close"] < row["BB_Lower"]:
117         score += 15
118     return min(score, 100)
119
120 def momentum_signal(self, df: pd.DataFrame) -> int:
121     """Returns +1 (bullish), -1 (bearish), or 0 (neutral) from 5-period momentum."""
122     m = float(df["Momentum"].iloc[-1])
123     if m > 0.02:
124         return 1
125     if m < -0.02:
126         return -1
127     return 0
128
129 def generate_signal(self, row: pd.Series, momentum: int) -> str:
130     if (
131         row["Close"] > row["SMA_20"]
132         and row["RSI"] < 70
133         and row["MACD"] > row["Signal_Line"]
134         and momentum >= 0
135     ):
136         return "STRONG BUY"
137     if (
138         row["Close"] < row["SMA_20"]
139         and row["RSI"] > 30
140         and row["MACD"] < row["Signal_Line"]
141         and momentum <= 0
142     ):
143         return "STRONG SELL"
144     return "HOLD"
145
146 def calculate_stops(
147     self, price: float, atr: float, signal: str
148 ) -> tuple[Optional[float], Optional[float]]:
149     """ATR-based stops: 2×ATR risk, 3×ATR reward – adapts to market volatility."""
150     if signal == "STRONG BUY":
151         return price - 2 * atr, price + 3 * atr
152     if signal == "STRONG SELL":
153         return price + 2 * atr, price - 3 * atr
154     return None, None
155
156 # ----- #
157 # Main entry point #
158 # ----- #
159
```

```
160     def analyze(self, ticker: str = "BTC-USD") -> TradeSignal:
161         df = self.fetch_data(ticker)
162         df = self.calculate_indicators(df)
163         latest = df.iloc[-1]
164
165         price = float(latest["Close"])
166         confidence = self.score_signal(latest)
167         momentum = self.momentum_signal(df)
168         signal = self.generate_signal(latest, momentum)
169         stop_loss, take_profit = self.calculate_stops(price, float(latest["ATR"]), signal)
170
171         result = TradeSignal(
172             ticker=ticker,
173             signal=signal,
174             price=price,
175             confidence=confidence,
176             stop_loss=stop_loss,
177             take_profit=take_profit,
178             timestamp=datetime.now().strftime("%Y-%m-%d %H:%M"),
179         )
180         self._print_report(result)
181         return result
182
183     def _print_report(self, r: TradeSignal) -> None:
184         sep = "=" * 50
185         print(sep)
186         print(f"    {self.name}")
187         print(sep)
188         print(f"    Ticker:      {r.ticker}")
189         print(f"    Price:       ${r.price:,.2f}")
190         print(f"    Time:        {r.timestamp}")
191         print(f"    Signal:      {r.signal}")
192         print(f"    Confidence:  {r.confidence}/100")
193         if r.stop_loss is not None:
194             print(f"    Stop-Loss:   ${r.stop_loss:,.2f}")
195             print(f"    Take-Profit: ${r.take_profit:,.2f}")
196         print(sep)
197         print("    WARNING: Educational use only. Not financial advice.\n")
198
199
200 # ----- #
201 # Auto-run loop                                     #
202 # ----- #
203
204 def clear_screen() -> None:
205     """Cross-platform screen clear (Mac/Linux and Windows)."""
206     os.system("cls" if os.name == "nt" else "clear")
207
208
209 if __name__ == "__main__":
210     bot = TradingBot()
211     print("Bot is now running automatically every 5 minutes.")
212     print("Press Ctrl+C to stop.\n")
```

```
213
214     while True:
215         try:
216             clear_screen()
217             bot.analyze("BTC-USD")    # Change ticker here e.g. "ETH-USD", "AAPL"
218             print("Next check in 5 minutes...\n")
219             time.sleep(300)
220         except KeyboardInterrupt:
221             print("\nBot stopped. See you later!")
222             break
223         except Exception as e:
224             print(f"Error: {e}")
225             print("Retrying in 60 seconds...\n")
226             time.sleep(60)
227
```